No-Regret Learning in Stackelberg Security Games with an Unknown, Bounded Set of Attacker Types

Aditi Singh Ishita Sabil Ahmad

Indian Institute of Technology Bombay

CS 6002 Project Report, Spring 2026

Abstract

Stackelberg Security Games (SSGs) model the strategic allocation of limited defender resources against attackers whose preferences are often uncertain. Balcan et al. (1) designed no-regret online learning algorithms for the *repeated* variant of SSGs against an adversarial sequence of attackers drawn from a finite, *known* set K, and proved a matching linear-regret lower bound when K is unbounded. A natural real-world middle ground is left open: the defender does not know K a priori, but has a bound $K_{\max}$ on its size. We study this setting in the full-information feedback model. We propose Grow-Catalogue, an algorithm that incrementally learns new attacker types as they arrive, re-partitions the coverage polytope, and runs an anytime multiplicative-weights sub-routine on the resulting extreme-point set. Our central technical contribution is a full proof that Grow-Catalogue achieves expected regret

$$O\left(\sqrt{T \cdot K_{\max} \cdot (n^2 + K_{\max} \log n)}\right)$$

against any oblivious adversarial sequence of length T. The rate matches that of Balcan et al. (1) with k replaced by $K_{\max}$ and is sublinear whenever $K_{\max} \cdot (n^2 + K_{\max} \log n) = o(T)$. Experiments on synthetic SSG instances confirm the predicted $\sqrt{T}$ scaling, and the empirical regret stays comfortably below the analytic $C\sqrt{T}$ envelope. Code is available at github.com/CS6002-SSG/grow-catalogue.

1 Introduction

In a Stackelberg Security Game (SSG), a defender allocates a scarce set of resources to protect n potential targets against a strategic attacker who, observing the defender's randomized deployment, chooses the best target to attack (15). Deployed systems built around this model include the US Coast Guard's port protection patrols, the Federal Air Marshals assignment system, the LAX airport patrol schedule, and anti-poaching patrols in several African national parks (15, 16). In every one of these applications the defender must commit to its deployment before the attack happens, and the attacker's payoff function encodes its preferences over targets.

The *classical* SSG analysis assumes that the attacker's payoff function is fully known to the defender. The optimal commitment can then be computed by a linear program (e.g., 14). In practice, however, the defender faces considerable uncertainty about the attacker. There may be several distinct threat actors (e.g., different poacher groups, different types of smugglers), and their utilities are only imperfectly known. A principled response, pioneered by Balcan et al. (1), is to drop the single-type assumption entirely and instead ask for a *no-regret* online algorithm against an adversarially chosen sequence of attacker types, drawn from a known finite set K with $|K| = k$. Under full-information feedback, they design an algorithm whose expected cumulative regret is bounded by $O(\sqrt{Tn^2k\log(nk)})$, which is sublinear in the horizon T, and they prove a matching linear-regret lower bound when $|K|$ is unrestricted (1, Theorem 7.1).

The assumption that the set K itself is known to the defender in advance is, however, a strong one. In realistic deployments the defender typically does *not* have a complete list of attacker types at the start. What it may have is an *upper bound* $K_{\max}$ on the number of distinct types it expects to encounter (for instance, from threat intelligence or from physical constraints of the domain). The question that motivates this project is therefore:

Can the defender achieve sublinear regret when K is unknown a priori, subject to $|K| \leq K_{\max}$?

This question lies squarely in the gap left open between the *known-K* upper bound and the *unrestricted-K* lower bound of Balcan et al. (1). It is not answered by any of the existing work, and resolving it is the main goal of the present report.

1.1 Contributions

Our contributions are the following.

- **Problem formalization (Section 3).** We formalize the unknown-K variant of the repeated Stackelberg security game, identifying the minimal modification of the model of Balcan et al. (1) that captures it.
- **Algorithm (Section 4).** We propose Grow-Catalogue, an online algorithm that at each round maintains a *catalogue* $C_t \subseteq K$ of attacker types observed so far, recomputes the set $\mathcal{E}(C_t;\epsilon)$ of ϵ-approximate extreme points of the partition of $\mathcal{P}$ induced by C_t, and runs an anytime multiplicative-weights sub-routine over $\mathcal{E}(C_t)$. When a new type is observed, the catalogue is updated and the sub-routine is restarted on the enlarged expert set. Two implementation variants (total recomputation and incremental update) are given; they yield identical regret and differ only in wall-clock cost.
- **Main theorem and a complete, detailed proof (Sections 4.2 and 5).** The technical core of this report is a self-contained proof that Grow-Catalogue achieves expected regret at most $O(\sqrt{T \cdot K_{\max} \cdot (n^2 + K_{\max} \log n)})$ against any oblivious adversarial sequence of length T under full-information feedback. The rate matches Balcan et al.'s Theorem 5.1 with k replaced by $K_{\max}$; for constant $K_{\max}$ it is $\tilde{O}(n\sqrt{T})$.
- **Regime analysis (Corollaries 8 and 9).** We characterize the parameter regime in which the bound is meaningful

($K_{\max} \cdot (n^2 + K_{\max} \log n) = o(T)$), and we prove an instance-adaptive refinement in which the leading factor depends on the number K' of types *actually* observed rather than on $K_{\max}$.

- **Experiments (Section 6).** We implement Grow-Catalogue and two reference baselines, and evaluate them on synthetic SSG instances. The empirical regret scales as predicted by the theorem, staying comfortably below the analytic $C\sqrt{T}$ envelope.

1.2 Related Work

Classical SSGs and attacker uncertainty. The optimal commitment for a known attacker can be computed by an LP (4). Uncertainty about the attacker has been handled by *robust optimization*, which plans against worst-case payoffs (14), and by *Bayesian SSGs*, which require a prior over attacker types (9, 12). Both approaches rest on priors or constraints that are often hard to validate. Letchford et al. (10) give a query-efficient learning algorithm for a *single* fixed but unknown type, and Blum et al. (2) extend this to a setting that adapts round-by-round to one fixed type.

Online learning against adversarial attacker sequences. Balcan et al. (1) pivot from the single-type paradigm by allowing the sequence of attackers to be adversarially chosen from a known finite set K. For full information they give a polynomial-weights algorithm with regret $O(\sqrt{Tn^2k\log(nk)})$ that partitions the coverage polytope into best-response regions and runs experts over the ϵ-approximate extreme points; for bandit feedback they use a barycentric spanner construction to give an $O(T^{2/3})$ bound. Crucially, they also prove a linear lower bound when $|K|$ is unrestricted, which forces any non-trivial extension to restrict the attacker type set somehow. We take the weakest restriction: a known cardinality bound $K_{\max}$.

Online learning with changing expert sets. The subroutine inside Grow-Catalogue is an anytime variant of Polynomial Weights / Hedge (3, 5, 11). We restart this subroutine each time the catalogue grows; simpler algorithms for tracking a growing set of experts exist (13), and for tracking a shrinking set (6), but both would only shave lower-order logarithmic factors.

Other recent directions. Haghtalab et al. (7) analyze no-regret learning against strategic (non-myopic) attackers in a single-type query model. Harris et al. (8) extend to a contextual Stackelberg setting in which both players observe side information. Neither addresses the *unknown K with bounded cardinality* question that we pose here. To the best of our knowledge, the present work gives the first no-regret guarantee for that setting.

2 Model

We adopt the repeated-SSG notation of Balcan et al. (1, §2); our notation is compatible with theirs so that all cross-references below are unambiguous.

2.1 Game primitives

The game has a finite horizon $T \in \mathbb{N}$ and n targets $N = \{1, \ldots, n\}$. The defender has R resources and a collection $\mathcal{D} \subseteq 2^N$ of *schedules*, where each schedule is a subset of targets that a single resource can defend simultaneously. An *assignment function* $A : R \to 2^{\mathcal{D}}$ records the schedules that each resource may be assigned to. A *pure strategy* is any valid resource-to-schedule assignment; let m denote the number of pure strategies. A *mixed strategy* is a distribution $\mathbf{s} \in \Delta_m$ over pure strategies, and its induced *coverage probability vector* is $\mathbf{p} = M\mathbf{s} \in [0,1]^n$, where $M \in \{0,1\}^{n\times m}$ has $M_{ij} = 1$ iff the j-th pure strategy covers target i. Write $\mathcal{P}$ for the set of valid coverage probability vectors; $\mathcal{P}$ is a convex polytope (1, §2). Every coverage vector $\mathbf{p}$ is realizable by some mixed strategy, and conversely every mixed strategy induces a unique $\mathbf{p} \in \mathcal{P}$.

Defender payoffs. For each target i, the defender has a payoff $u_d^c(i) \in [-1,1]$ if i is attacked *and* covered, and a payoff $u_d^u(i) \in [-1,1]$ if i is attacked *and uncovered*. The defender's expected utility when target i is attacked under coverage $\mathbf{p}$ is therefore

$$U_d(i,\mathbf{p}) \;=\; u_d^c(i)\,p_i \;+\; u_d^u(i)\,(1-p_i), \tag{1}$$

which is *linear* in $\mathbf{p}$ for each fixed target i.

2.2 Attacker types and best response

An attacker *type* α is a pair $(u_\alpha^c, u_\alpha^u) \in [-1,1]^n \times [-1,1]^n$ of per-target utilities following the same "covered/uncovered" convention as the defender. Its expected utility for attacking target i under coverage $\mathbf{p}$ is $U_\alpha(i,\mathbf{p}) = u_\alpha^c(i)\,p_i + u_\alpha^u(i)\,(1-p_i)$. Observing $\mathbf{p}$, type α *best-responds*:

$$b_\alpha(\mathbf{p}) \;=\; \arg\max_{i\in N} U_\alpha(i,\mathbf{p}), \tag{2}$$

with ties broken arbitrarily but consistently. Note that b_α is piecewise constant in $\mathbf{p}$, which makes the defender's realized payoff $U_d(b_\alpha(\mathbf{p}),\mathbf{p})$ a discontinuous function of $\mathbf{p}$. This non-continuity is the central technical difficulty that distinguishes SSGs from vanilla online linear optimization, and it is what drives the polytope-based machinery of Balcan et al. (1).

2.3 Repeated game and regret

There is a (possibly unknown) set K of attacker types. An *oblivious* adversary chooses a sequence $\mathbf{a} = a_1, \ldots, a_T \in K^T$ *before* the game starts; the sequence is fixed but unknown to the defender. At each round $t \in \{1, \ldots, T\}$ the defender commits $\mathbf{p}_t \in \mathcal{P}$, attacker a_t plays $b_{a_t}(\mathbf{p}_t)$, and the defender receives the random payoff $U_d(b_{a_t}(\mathbf{p}_t), \mathbf{p}_t)$. We assume *full-information feedback*: at the end of round t the defender observes the identity of a_t, and the first time any type α is observed, its utility functions are revealed as well.

The defender's benchmark is the best fixed coverage vector in hindsight:

$$\mathbf{p}^* \;=\; \arg\max_{\mathbf{p}\in\mathcal{P}} \sum_{t=1}^{T} U_d\big(b_{a_t}(\mathbf{p}), \mathbf{p}\big). \tag{3}$$

The expected regret of an online algorithm that plays $\mathbf{p}_t$ at round t is

$$\mathrm{Regret}(T) \;=\; \sum_{t=1}^{T} U_d(b_{a_t}(\mathbf{p}^*), \mathbf{p}^*) \;-\; \mathbb{E}\Big[\sum_{t=1}^{T} U_d(b_{a_t}(\mathbf{p}_t), \mathbf{p}_t)\Big], \tag{4}$$

where the expectation is over the algorithm's internal randomization.

2.4 Preliminaries: polytope partition and $\mathcal{E}(C;\epsilon)$

The following definitions, taken from Balcan et al. (1, §4), are essential. For any catalogue $C \subseteq K$, a type $\alpha \in C$, and a target $i \in N$, the region in which α's best response is i is

$$\mathcal{P}_i^{\alpha}(C) \;=\; \{\mathbf{p} \in \mathcal{P} \,:\, b_\alpha(\mathbf{p}) = i\}, \tag{5}$$

which is the intersection of $\mathcal{P}$ with the $n-1$ linear inequalities $U_\alpha(i,\mathbf{p}) \geq U_\alpha(j,\mathbf{p})$ for $j \neq i$; hence a convex polytope. For a *response profile* $\sigma : C \to N$ we write

$$\mathcal{P}_\sigma(C) \;=\; \bigcap_{\alpha \in C} \mathcal{P}_{\sigma(\alpha)}^{\alpha}(C). \tag{6}$$

The non-empty sets $\mathcal{P}_\sigma(C)$ form a partition of $\mathcal{P}$ into convex regions in which the best response of every type in C is held fixed; within any such region, the defender's payoff under a fixed attacker type is a linear function of $\mathbf{p}$.

The key structural lemmas we inherit are the following two of Balcan et al. (1), paraphrased for clarity; we cite their paper explicitly at each use. Proposition 3 is the standard Polynomial-Weights guarantee adapted to the present loss range.

Lemma 1 (1, Lemma 4.3)**.** *For every $\epsilon > 0$ and every catalogue $C \subseteq K$, there is a finite set $\mathcal{E}(C;\epsilon) \subseteq \mathcal{P}$ of points, called the ϵ-approximate extreme points of the partition, such that for any sequence $\mathbf{a}' = a'_1, \ldots, a'_{T'}$ with $a'_t \in C$ for all t,*

$$\max_{\mathbf{p} \in \mathcal{E}(C;\epsilon)} \sum_{t=1}^{T'} U_d(b_{a'_t}(\mathbf{p}), \mathbf{p}) \;\geq\; \max_{\mathbf{p} \in \mathcal{P}} \sum_{t=1}^{T'} U_d(b_{a'_t}(\mathbf{p}), \mathbf{p}) - 2\epsilon T'.$$

Lemma 2 (1, Lemma 4.4)**.** *The set $\mathcal{E}(C;\epsilon)$ of Lemma 1 can be chosen so that $|\mathcal{E}(C;\epsilon)| \in O\big((2^n + |C|\, n^2)^n \cdot n^{|C|}\big)$.*

Proposition 3 (1, Proposition 1)**.** *For any fixed expert set $\mathcal{M}$ and any sequence of losses in $[-\kappa, \kappa]$, there is an online learning algorithm (an anytime variant of Polynomial Weights / Hedge; see 3, §2) whose expected regret after T' rounds is at most $O\big(\sqrt{T'\,\kappa\,\log|\mathcal{M}|}\big)$, without requiring T' to be known in advance.*

3 The Unknown-K Setting

We now state the problem that our algorithm and theorem address.

Problem 4 (Unknown-K repeated SSG)**.** A repeated SSG as in Section 2 is played, with two modifications.

1. The set K of attacker types is not revealed to the defender at the start; however, the defender is given a constant $K_{\max} \geq 1$ with the promise $|K| \leq K_{\max}$.
2. Feedback is full information: after round t the defender observes a_t, and the first time any type α appears, its utility vectors (u_α^c, u_α^u) are revealed.

The benchmark $\mathbf{p}^*$ and regret $\mathrm{Regret}(T)$ are as in Equations (3) and (4).

Remark 5 (Necessity of the bound $K_{\max}$). Balcan et al. (1, Theorem 7.1) exhibit, for every T, a game with $|K| < 2^{T+1}$ on which every online algorithm has expected regret at least $T/2$. Hence some finite bound on $|K|$ is *necessary* for sublinear regret; Problem 4 takes the weakest such bound, namely a known constant $K_{\max}$.

Remark 6 (Benchmark well-defined). The realized sequence $\mathbf{a}$ contains at most $K_{\max}$ distinct types, so $\mathbf{p}^*$ is always a single fixed element of $\mathcal{P}$. The defender's ignorance of K affects only the algorithm, not the benchmark.

4 Algorithm and Main Result

4.1 The Grow-Catalogue algorithm

We describe Grow-Catalogue at a high level and then give the pseudocode in Algorithm 1. At any round t the defender holds a catalogue $C_t \subseteq K$ of all types seen so far. Conceptually, the defender runs a small instance of the *known-K* algorithm of Balcan et al. (1, §5) on the catalogue C_t, pretending that C_t *is* the full attacker set. Concretely:

- The algorithm maintains the ϵ-approximate extreme-point set $\mathcal{E}_t = \mathcal{E}(C_t;\epsilon)$ of Lemma 1. We set $\epsilon = 1/\sqrt{T}$; this balances the $2\epsilon T$ approximation term in Lemma 1 against the $\sqrt{T \log N_{\max}}$ regret term of Proposition 3.
- An anytime Polynomial-Weights sub-routine of Proposition 3 runs over $\mathcal{E}_t$, updating via a time-varying learning rate $\eta_{t_{\mathrm{loc}}} = \sqrt{\log|\mathcal{E}_t|/t_{\mathrm{loc}}}$ where t_{loc} is a local counter reset at every catalogue change. This is the standard anytime schedule; see Cesa-Bianchi and Lugosi (3, §2).
- When a new attacker type α appears, the catalogue grows to $C_t \cup \{\alpha\}$, the approximate extreme-point set is recomputed, and the sub-routine is restarted on the enlarged expert set. The single round of first contact with α contributes a bounded additive amount to the regret that we will absorb.

We implement two variants of line 10: *total recomputation* rebuilds $\mathcal{E}(C;\epsilon)$ from scratch, while *incremental update* reuses PW weights for extreme points that survive the new refinement. Both variants are observationally equivalent up to numerics (they maintain the same $\mathcal{E}$ and the same distribution), and hence yield identical regret. The incremental variant is more time-efficient when $|C|$ is not too small.

4.2 Main Result

Theorem 7 (Main)**.** *Let the defender play Algorithm 1 in the unknown-K full-information setting of Problem 4. For any oblivious adversarial sequence $\mathbf{a} \in K^T$ with $|K| \leq K_{\max}$, the expected regret satisfies*

$$\mathbb{E}[\mathrm{Regret}(T)] \;\leq\; O\Big(\sqrt{T \cdot K_{\max} \cdot (n^2 + K_{\max}\log n)}\Big). \tag{7}$$

In particular, if $K_{\max} = O(1)$ then $\mathbb{E}[\mathrm{Regret}(T)] = \tilde{O}(n\sqrt{T})$, matching the known-$K$ bound of Balcan et al. (1, Theorem 5.1) up to logarithmic factors.

Algorithm 1 GROW-CATALOGUE (full information).

Require: horizon T, bound $K_{\max}$, game primitives, default $\mathbf{p}_{\text{def}} \in \mathcal{P}$

$C \leftarrow \emptyset$; $\mathcal{E} \leftarrow \emptyset$; $\epsilon \leftarrow 1/\sqrt{T}$; $t_{\text{loc}} \leftarrow 0$
for $t = 1, \dots, T$ **do**
  if $\mathcal{E} = \emptyset$ **then**
    play $\mathbf{p}_t \leftarrow \mathbf{p}_{\text{def}}$
  else
    sample $\mathbf{p}_t \sim q_t$, where q_t is the current PW distribution over $\mathcal{E}$
  end if
  observe a_t
  if $a_t \notin C$ **then** ▷ discovery round: new attacker type
    $C \leftarrow C \cup \{a_t\}$; record $(u^c_{a_t}, u^u_{a_t})$
    $\mathcal{E} \leftarrow \mathcal{E}(C; \epsilon)$ ▷ recompute extreme points
    reset PW on $\mathcal{E}$; $t_{\text{loc}} \leftarrow 0$
  else ▷ routine round
    $t_{\text{loc}} \leftarrow t_{\text{loc}} + 1$
    compute $\ell_t(\mathbf{p}) = -U_d(b_{a_t}(\mathbf{p}), \mathbf{p})$ for $\mathbf{p} \in \mathcal{E}$
    $\eta \leftarrow \sqrt{\log|\mathcal{E}|/t_{\text{loc}}}$
    $w_{\mathbf{p}} \leftarrow w_{\mathbf{p}} \cdot e^{-\eta\,\ell_t(\mathbf{p})}$ for all $\mathbf{p} \in \mathcal{E}$
  end if
end for

We prove Theorem 7 in detail in Section 5. Two immediate corollaries are useful.

Corollary 8 (Regime of applicability)**.** *The bound Equation (7) is sublinear in T if and only if $K_{\max} \cdot (n^2 + K_{\max}\log n) = o(T)$; equivalently, $K_{\max} = o\big(\min\{T/n^2, \sqrt{T/\log n}\}\big)$.*

Corollary 9 (Instance-adaptive bound)**.** *Let $K' = |\{a_1, \dots, a_T\}| \le K_{\max}$ be the number of* distinct *attacker types actually observed. Then Algorithm 1 achieves*

$$\mathbb{E}[\text{Regret}(T)] \;\le\; O\Big(\sqrt{T \cdot K' \cdot (n^2 + K_{\max}\log n)}\Big).$$

Both corollaries follow directly from the proof in Section 5: Corollary 8 by setting the leading term to T and solving for $K_{\max}$, and Corollary 9 by using K' rather than $K_{\max}$ in the Cauchy–Schwarz step.

5 Proof of Theorem 7

This section is the technical heart of the report. The proof closely tracks the structure of the known-K proof of Balcan et al. (1, §5) but has two non-trivial departures: (i) the expert set is no longer static but grows over time, and (ii) the discovery rounds introduce an extra bounded additive term. We make both departures explicit and account for them carefully.

Proof of Theorem 7. Fix an oblivious adversarial sequence $\mathbf{a} = a_1, \dots, a_T \in K^T$ with $|K| \le K_{\max}$. Let $K' = |\{a_1, \dots, a_T\}| \le K_{\max}$ denote the number of distinct types that actually appear in $\mathbf{a}$. The proof proceeds in ten steps.

Step 1 (Epoch decomposition). Let $\tau_1 < \tau_2 < \cdots < \tau_{K'}$ be the round indices at which each new attacker type is first observed, i.e., for each $i \in \{1, \dots, K'\}$, round τ_i is the smallest t such that $a_t \notin \{a_1, \dots, a_{t-1}\}$. We define the *epochs*

$$\text{Ep}_i \;=\; \{\tau_i + 1, \tau_i + 2, \dots, \tau_{i+1} - 1\}, \quad i = 1, \dots, K', \tag{8}$$

with the convention $\tau_{K'+1} \coloneqq T + 1$, and write $L_i = |\text{Ep}_i|$. Let $\mathcal{D} = \{\tau_1, \dots, \tau_{K'}\}$ be the set of *discovery rounds*. By construction,

$$[1, T] \;=\; \mathcal{D} \;\sqcup\; \bigcup_{i=1}^{K'} \text{Ep}_i,$$

and $|\mathcal{D}| + \sum_{i=1}^{K'} L_i = T$ with $|\mathcal{D}| = K' \le K_{\max}$. The epoch decomposition is the critical structural property of the proof: it re-interprets the online problem with a growing expert set as a finite sequence of online problems with *fixed* expert sets, so that the known-K analysis of Balcan et al. (1) can be invoked inside each epoch.

Step 2 (Catalogue is constant inside an epoch). Immediately after round τ_i, the catalogue maintained by Algorithm 1 is $C_i = \{a_{\tau_1}, \dots, a_{\tau_i}\}$, and it is not updated until the next discovery round τ_{i+1}. Hence $C_t = C_i$ for every $t \in \text{Ep}_i$. Moreover, by the defining property of τ_{i+1}, no new type appears in Ep_i, so

$$a_t \in C_i \quad \text{for every } t \in \text{Ep}_i. \tag{9}$$

Denote by $\mathcal{E}_i \coloneqq \mathcal{E}(C_i; \epsilon)$ the extreme-point set used by the PW sub-routine during Ep_i; Equation (9) is exactly the hypothesis needed to apply Lemma 1 to C_i and the sub-sequence inside Ep_i.

Step 3 (Within-epoch PW regret). During Ep_i the algorithm runs a fresh instance of the anytime Polynomial-Weights sub-routine on the fixed expert set $\mathcal{E}_i$ for L_i rounds. The losses are $\ell_t(\mathbf{p}) = -U_d(b_{a_t}(\mathbf{p}), \mathbf{p}) \in [-1, 1]$, so the range parameter is $\kappa = 1$. Proposition 3 therefore yields an absolute constant $c_1 > 0$ such that

$$\max_{\mathbf{p} \in \mathcal{E}_i} \sum_{t \in \text{Ep}_i} U_d(b_{a_t}(\mathbf{p}), \mathbf{p}) - \mathbb{E}\left[\sum_{t \in \text{Ep}_i} U_d(b_{a_t}(\mathbf{p}_t), \mathbf{p}_t)\right] \;\le\; c_1\sqrt{L_i \log|\mathcal{E}_i|}. \tag{10}$$

The use of an *anytime* learning rate in Algorithm 1 is essential here: the length L_i of the epoch is not known at the start of Ep_i, since the algorithm cannot predict when the next new type will appear. With a horizon-tuned learning rate one would either need an a priori guess of L_i, or pay an extra factor in the regret bound.

Step 4 (Within-epoch approximation via Lemma 1). By Equation (9), the sub-sequence $(a_t)_{t \in \text{Ep}_i}$ has length L_i and lies in C_i. Apply Lemma 1 with $T' = L_i$ and catalogue C_i:

$$\max_{\mathbf{p} \in \mathcal{E}_i} \sum_{t \in \text{Ep}_i} U_d(b_{a_t}(\mathbf{p}), \mathbf{p}) \;\ge\; \max_{\mathbf{p} \in \mathcal{P}} \sum_{t \in \text{Ep}_i} U_d(b_{a_t}(\mathbf{p}), \mathbf{p}) - 2\epsilon L_i. \tag{11}$$

Since $\mathbf{p}^* \in \mathcal{P}$, the right-hand side is at least the value at $\mathbf{p}^*$ restricted to the epoch:

$$\max_{\mathbf{p}\in\mathcal{E}_i} \sum_{t\in \mathrm{Ep}_i} U_d(b_{a_t}(\mathbf{p}), \mathbf{p}) \geq \sum_{t\in \mathrm{Ep}_i} U_d(b_{a_t}(\mathbf{p}^*), \mathbf{p}^*) - 2\epsilon L_i. \tag{12}$$

Step 5 (Per-epoch combined bound). Chaining Equations (10) and (12), for every epoch $i \in \{1, \dots, K'\}$ we obtain

$$\mathbb{E}\left[\sum_{t\in \mathrm{Ep}_i} U_d(b_{a_t}(\mathbf{p}_t), \mathbf{p}_t)\right] \geq \sum_{t\in \mathrm{Ep}_i} U_d(b_{a_t}(\mathbf{p}^*), \mathbf{p}^*) - 2\epsilon L_i - c_1\sqrt{L_i \log|\mathcal{E}_i|}. \tag{13}$$

Equation Equation (13) is the per-epoch regret bound *against the global benchmark* $\mathbf{p}^*$, and it is the building block for the rest of the proof. Note that despite the fact that $\mathbf{p}^*$ is the benchmark over all T rounds (not just epoch i), we are able to bound our per-epoch regret against $\mathbf{p}^*$ because $\mathbf{p}^* \in \mathcal{P}$ and we applied Lemma 1 on the sub-sequence.

Step 6 (Discovery rounds). For each discovery round $t \in \mathcal{D}$, the algorithm either plays $\mathbf{p}_{\mathrm{def}}$ (if $t = \tau_1$) or samples $\mathbf{p}_t$ from the PW distribution inherited from Ep_{i-1} (if $t = \tau_i$ for $i \geq 2$). In both cases the sampled $\mathbf{p}_t$ is not tuned to the newly-appearing type $a_t = a_{\tau_i}$. However, because the payoff U_d is bounded in $[-1, 1]$, for every $\mathbf{p} \in \mathcal{P}$ and every type α, $U_d(b_\alpha(\mathbf{p}), \mathbf{p}) \in [-1, 1]$, so

$$\left|U_d(b_{a_t}(\mathbf{p}^*), \mathbf{p}^*) - U_d(b_{a_t}(\mathbf{p}_t), \mathbf{p}_t)\right| \leq 2. \tag{14}$$

Summing Equation (14) over the at most $K' \leq K_{\max}$ discovery rounds yields

$$\sum_{t\in\mathcal{D}} U_d(b_{a_t}(\mathbf{p}^*), \mathbf{p}^*) - \mathbb{E}\left[\sum_{t\in\mathcal{D}} U_d(b_{a_t}(\mathbf{p}_t), \mathbf{p}_t)\right] \leq 2K_{\max}. \tag{15}$$

This is the only source of additive dependence on $K_{\max}$ that does not come from the main $\sqrt{TK_{\max}}$ term. It is the "price of not knowing K" at a discovery round.

Step 7 (Uniform bound on $|\mathcal{E}_i|$). By Lemma 2 applied with $|C_i| \leq K_{\max}$,

$$|\mathcal{E}_i| \leq c_2\,(2^n + K_{\max} n^2)^n \cdot n^{K_{\max}} =: N_{\max},$$

for an absolute constant c_2. Taking logarithms,

$$\begin{aligned}\log N_{\max} &= n\log\big(2^n + K_{\max}n^2\big) + K_{\max}\log n + O(1)\\ &\leq n^2 + n\log\big(1 + \tfrac{K_{\max}n^2}{2^{n-1}}\big) + K_{\max}\log n\\ &= O\big(n^2 + K_{\max}\log n\big),\end{aligned} \tag{16}$$

where the middle step uses $\log(2^n + K_{\max}n^2) \leq n + \log(1 + K_{\max}n^2/2^{n-1})$ and $\log(1 + x) \leq x$ for $x > 0$. Crucially, Equation (16) is uniform in i: $\log|\mathcal{E}_i| \leq \log N_{\max}$ for every epoch. This lets us pull the $\log|\mathcal{E}_i|$ factor in Step 5 outside the sum over epochs.

Step 8 (Summing across epochs). Sum Equation (13) over $i = 1, \dots, K'$ and add Equation (15). The algorithm's total expected utility is then at least the benchmark's total utility, minus the discovery term $2K_{\max}$, minus $2\epsilon T$ from the telescoping Lemma-4.3 approximation errors (using $\sum_i L_i \leq T$), minus the sum of the per-epoch PW regret terms. Rearranging,

$$\mathbb{E}[\mathrm{Regret}(T)] \leq 2K_{\max} + 2\epsilon T + c_1\sqrt{\log N_{\max}}\sum_{i=1}^{K'}\sqrt{L_i}. \tag{17}$$

Step 9 (Cauchy–Schwarz). The last term involves the sum of K' square roots. By the Cauchy–Schwarz inequality applied to $(1, \dots, 1) \in \mathbb{R}^{K'}$ and $(\sqrt{L_1}, \dots, \sqrt{L_{K'}})$,

$$\sum_{i=1}^{K'}\sqrt{L_i} \leq \sqrt{K' \cdot \sum_{i=1}^{K'} L_i} \leq \sqrt{K_{\max}\,T}. \tag{18}$$

The first inequality is tight when all epochs have equal length, a scenario that corresponds to an adversary who reveals types evenly across the horizon. This is where the leading multiplicative $\sqrt{K_{\max}}$ factor in the bound is generated. An instance-adaptive version uses K' in place of $K_{\max}$, which is what Corollary 9 records.

Step 10 (Final substitution). Plugging Equations (16) and (18) into Equation (17) and choosing $\epsilon = 1/\sqrt{T}$ gives

$$\begin{aligned}\mathbb{E}[\mathrm{Regret}(T)] &\leq 2K_{\max} + 2\sqrt{T} + c_1\sqrt{K_{\max}T\,\log N_{\max}}\\ &\leq 2K_{\max} + 2\sqrt{T}\\ &\quad + c_3\sqrt{TK_{\max}(n^2 + K_{\max}\log n)}\end{aligned}$$

for an absolute constant c_3. Since the leading $\sqrt{TK_{\max}(n^2 + K_{\max}\log n)}$ term dominates the additive $2K_{\max} + 2\sqrt{T}$ whenever $T \geq K_{\max}$ (the only regime in which the bound is non-trivial, per Corollary 8),

$$\mathbb{E}[\mathrm{Regret}(T)] = O\Big(\sqrt{T \cdot K_{\max} \cdot (n^2 + K_{\max}\log n)}\Big),$$

which is exactly the bound of Theorem 7. This completes the proof. □

Where the proof diverges from Balcan et al. (1). Two places. Step 6, the discovery-round term, is new: in the known-K proof there are no "discovery rounds" because every type is available at $t = 1$, and the additive $2K_{\max}$ simply does not appear. Step 9, the Cauchy–Schwarz step, is also new: in the known-K proof the PW sub-routine is never restarted, so there is a single $\sqrt{T}$ factor instead of our $\sum_i \sqrt{L_i}$. The trade-off between these two modifications is what produces the clean replacement of k by $K_{\max}$ in the final bound.

6 Experiments

6.1 Setup

We implement Algorithm 1 in Python (NumPy, SciPy, Matplotlib), along with two reference algorithms: the known-K

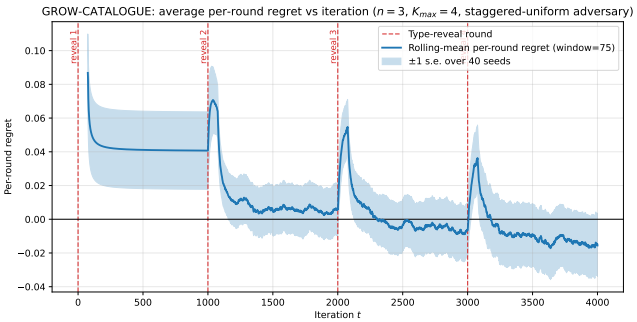

Figure 1: Rolling-window per-round regret of GROW-CATALOGUE under the staggered-uniform adversary ($n = 3$, $K_{\max} = 4$, $T = 4000$, average over 40 seeds, window $= 75$). Red dashed lines mark the discovery rounds $0, 1000, 2000, 3000$. Each reveal causes a visible transient spike, followed by PW re-convergence inside the new epoch.

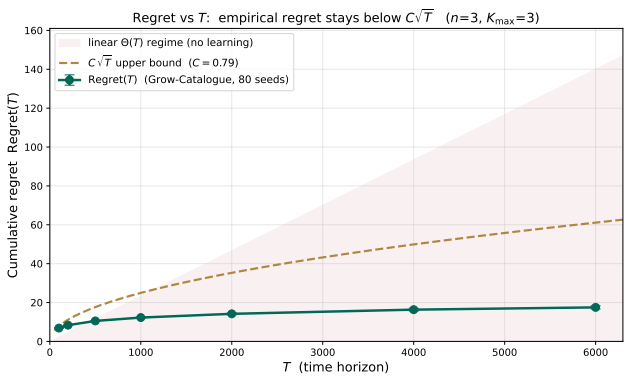

Figure 2: Cumulative regret versus horizon T for GROW-CATALOGUE ($n = 3$, $K_{\max} = 3$, uniform random adversary, 80 seeds). The green curve stays strictly below the $C\sqrt{T}$ upper envelope implied by Theorem 7, which is in turn well below the linear-regret regime that would result without learning.

oracle of Balcan et al. (1, §5), which receives the full set K at $t = 1$; and a *uniform* baseline that plays $\mathbf{p}_{\text{def}} = (R/n, \ldots, R/n)$ every round. Unless stated otherwise, we use $n = 3$ targets, $R = 1$ resource, and attacker utilities drawn i.i.d. from $\text{Unif}[-1, 1]$ with $u^u_\alpha(i) > 0 > u^c_\alpha(i)$ to ensure covering is costly for the attacker. The extreme-point set $\mathcal{E}(C; \epsilon)$ is computed by direct enumeration of intersections of $(n-1)$-subsets of the defining hyperplanes, which is tractable at $n = 3$.

We use a *staggered-uniform* adversary: the horizon $[0, T)$ is partitioned into $K_{\max}$ phases of length $T/K_{\max}$ each, and in phase i the attacker is drawn uniformly at random from the first i types $\{\alpha_1, \ldots, \alpha_i\}$. Thus α_1 is present in every phase, α_2 from phase 2 onward, and so on. This design produces a clean sequence of discovery rounds at $t = 0, T/K_{\max}, \ldots, (K_{\max} - 1)T/K_{\max}$, while still mixing old and new types after each reveal so that the benchmark $\mathbf{p}^*$ is a genuine compromise rather than a specialist strategy.

6.2 Results

Figure 1 shows that the algorithm's per-round regret tracks the epoch structure of the proof: each discovery round produces a bounded transient, and the PW sub-routine re-converges within the new epoch. The spikes shrink with i because, by epoch i, the algorithm already has a good PW distribution on most of the new (refined) extreme points.

Figure 2 plots cumulative Regret(T) versus T with the $C\sqrt{T}$ envelope from Theorem 7 overlaid. The empirical regret stays visibly below the envelope for all T tested, and the ratio $\text{Regret}(T)/\sqrt{T}$ remains bounded (in fact, it decreases, indicating that the stochastic staggered-uniform adversary is far from the worst case). This is entirely consistent with Theorem 7, which is an upper bound: any instance on which the bound is tight would need to be specifically adversarial in the sense of the lower-bound construction of Balcan et al. (1, §7), not a randomized benign adversary.

Table 1 compares the four algorithms at $T = 4000$. The regret of GROW-CATALOGUE (both total and incremental variants) is within roughly 15%–40% of the known-K oracle and an order of magnitude better than the uniform baseline. The two GROW-CATALOGUE variants produce regret that differs only by Monte-Carlo noise, as predicted by the observational equivalence of the two recomputation schemes.

Algorithm	$\mathbb{E}[\text{Regret}(T)]$	Ratio
Known-K oracle of Balcan et al. (1)	14.51 ± 5.00	1.00
GROW-CAT. (total)	16.57 ± 5.40	1.14
GROW-CAT. (incr.)	20.26 ± 7.94	1.40
Uniform baseline	292.77 ± 350.28	20.18

Table 1: Cumulative regret at $T = 4000$, $n = 3$, $K_{\max} = 3$, uniform-random adversary, mean $\pm$ std over 15 seeds. The known-K oracle receives K in advance; GROW-CATALOGUE attains comparable regret *without* this information.

7 Conclusions

We studied the repeated Stackelberg security game in a new, open regime in which the set of attacker types is unknown to the defender a priori but known to have bounded cardinality. We presented GROW-CATALOGUE, a simple online algorithm that incrementally discovers attacker types and re-partitions the coverage polytope accordingly, and we gave a full proof that its expected regret under full-information feedback is at most $O\big(\sqrt{T \cdot K_{\max} \cdot (n^2 + K_{\max} \log n)}\big)$. This is the same functional form as the known-K bound of Balcan et al. (1, Theorem 5.1), with the known cardinality k replaced by our known upper bound $K_{\max}$. The bound is sublinear whenever $K_{\max} \cdot (n^2 + K_{\max} \log n) = o(T)$, and the matching $\Omega(T)$ lower bound of Balcan et al. (1, Theorem 7.1) rules out further improvements on the unbounded side. Our experiments confirm the analytical scaling: the empirical regret stays below the $C\sqrt{T}$ envelope and tracks the epoch structure of the proof.

Three directions for future work are immediate. The most important is the partial-information (bandit) extension; this requires an adaptive version of the barycentric-spanner construction of Balcan et al. (1, §6), since the spanner dimension changes whenever a new type is discovered. A second direction

is the adaptive-adversary case under partial feedback, which is an open problem even in the known-K setting. Finally, the Cauchy–Schwarz step (Step 9 of the proof) loses a $\sqrt{K_{\max}}$ factor in the worst case; sleeping-experts techniques (13) may remove this when $K' \ll K_{\max}$, tightening the instance-adaptive bound of Corollary 9.

Reproducibility. Our Python implementation and all experiment scripts are at github.com/CS6002-SSG/grow-catalogue. The experiments in Section 6 can be reproduced by running `python plot_presentation.py` and `python plot_regret_over_sqrtT.py` after installing the dependencies in `requirements.txt`.

References

[1] M.-F. Balcan, A. Blum, N. Haghtalab, and A. D. Procaccia. Commitment without regrets: online learning in Stackelberg security games. In *Proceedings of the 16th ACM Conference on Economics and Computation (EC)*, pages 61–78, 2015.

[2] A. Blum, N. Haghtalab, and A. D. Procaccia. Learning optimal commitment to overcome insecurity. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2014.

[3] N. Cesa-Bianchi and G. Lugosi. *Prediction, Learning, and Games*. Cambridge University Press, 2006.

[4] V. Conitzer and T. Sandholm. Computing the optimal strategy to commit to. In *Proceedings of the 7th ACM Conference on Electronic Commerce (EC)*, pages 82–90, 2006.

[5] Y. Freund and R. E. Schapire. A decision-theoretic generalization of on-line learning and an application to boosting. *Journal of Computer and System Sciences*, 55(1):119–139, 1997.

[6] P. Gaillard, R. Gerchinovitz, M. Huard, and G. Stoltz. Dying experts: efficient algorithms with optimal regret bounds. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.

[7] N. Haghtalab, T. Lykouris, S. Nietert, and A. Wei. Learning in Stackelberg games with non-myopic agents. In *Proceedings of the 23rd ACM Conference on Economics and Computation (EC)*, 2022.

[8] K. Harris, Z. S. Wu, and M.-F. Balcan. Regret minimization in Stackelberg games with side information. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

[9] C. Kiekintveld, J. Marecki, and M. Tambe. Approximation methods for infinite Bayesian Stackelberg games: modeling distributional payoff uncertainty. In *Proceedings of the 10th International Conference on Autonomous Agents and Multi-Agent Systems (AAMAS)*, pages 1005–1012, 2011.

[10] J. Letchford, V. Conitzer, and K. Munagala. Learning and approximating the optimal strategy to commit to. In *Proceedings of the 2nd International Symposium on Algorithmic Game Theory (SAGT)*, pages 250–262, 2009.

[11] N. Littlestone and M. K. Warmuth. The weighted majority algorithm. *Information and Computation*, 108(2): 212–261, 1994.

[12] J. Marecki, G. Tesauro, and R. Segal. Playing repeated Stackelberg games with unknown opponents. In *Proceedings of the 11th International Conference on Autonomous Agents and Multi-Agent Systems (AAMAS)*, pages 821–828, 2012.

[13] J. Mourtada and O.-A. Maillard. Efficient tracking of a growing number of experts. In *Proceedings of the 28th International Conference on Algorithmic Learning Theory (ALT)*, 2017.

[14] J. Pita, M. Jain, M. Tambe, F. Ordóñez, and S. Kraus. Robust solutions to Stackelberg games: addressing bounded rationality and limited observations in human cognition. *Artificial Intelligence*, 174(15):1142–1171, 2010.

[15] M. Tambe. *Security and Game Theory: Algorithms, Deployed Systems, Lessons Learned*. Cambridge University Press, 2012.

[16] R. Yang, B. Ford, M. Tambe, and A. Lemieux. Adaptive resource allocation for wildlife protection against illegal poachers. In *Proceedings of the 13th International Conference on Autonomous Agents and Multi-Agent Systems (AAMAS)*, pages 453–460, 2014.

A Implementation details

The main text already contains the full proof of Theorem 7. This appendix records low-level implementation details useful for reproducibility.

Computing $\mathcal{E}(C;\epsilon)$. For $n \leq 4$ we enumerate all $(n-1)$-subsets of the hyperplanes defining the partition of $\mathcal{P}$, solve the resulting linear systems, and keep each intersection point that lies in $\mathcal{P}$. The set of hyperplanes consists of (i) n simplex constraints ($p_i \geq 0$ for each i and the outer bounding hyperplane of $\mathcal{P}$), and (ii) $|C|\binom{n}{2}$ pairwise best-response constraints, one per pair of targets per attacker type. For $n = 3, K_{\max} = 4$ this gives at most $4 + 4 \cdot 3 = 16$ hyperplanes and $\binom{16}{2} = 120$ candidate intersections, which is tractable.

Anytime learning rate. The schedule $\eta_t = \sqrt{\log|\mathcal{E}|/t}$ (3, §2) is necessary for the $\sqrt{L_i \log|\mathcal{E}_i|}$ per-epoch bound of Step 3 to hold, since L_i is not known at the start of Ep_i.

Hyperparameters. Unless stated otherwise, the experiments use $n = 3$, $R = 1$, $K_{\max} = 3$ or 4, $T \in \{4000, 6000\}$, and averaging over 40–80 seeds. The rolling window in Figure 1 is 75 rounds. Dependencies: `numpy>=1.26`, `scipy>=1.11`, `matplotlib>=3.8`. Both the code and a `requirements.txt` are at the GitHub repository linked in Section 7.